

Relatório 2º Projeto ASA 2025/2026

Grupo: AL030

Alunos: Leonor Costa Guedes (113396) e Manuel Francisco Santos Ramos Soares (113402)

Descrição do Problema e da Solução

O problema consiste em determinar todas as rotas entre pares de nós num grafo direcionado acíclico (DAG) cujo número de caminhos distintos módulo M esteja dentro de um intervalo $[m_1, m_2]$, organizando-as por "camião" de acordo com esse número. Trata-se de um problema de contagem de caminhos em DAG's com filtragem por modularidade e agrupamento por intervalo.

A solução proposta utiliza ordenação topológica seguida de programação dinâmica para contar caminhos a partir de cada nó origem. Para cada origem, calcula-se o número de caminhos para todos os outros nós (módulo M), filtra-se os pares cujo valor pertence ao intervalo especificado e agrupa-se os resultados por caminhão. Finalmente, ordenam-se lexicograficamente as rotas de cada caminhão para garantir a saída no formato exigido.

Análise Teórica

- **Leitura dos dados de entrada:** leitura de N , M , m_1 , m_2 , K e K pares de arestas.

Complexidade: $O(K)$

- **Construção do grafo:** inicialização das estruturas de dados e preenchimento da lista de adjacências. Cada aresta é processada uma vez.

Complexidade: $O(N + K)$

- **Aplicação do algoritmo de Khan:** contagem de caminhos para cada nó origem usando programação dinâmica após ordenação topológica.

Complexidade: $O(N^2 + NK)$

Complexidade global da solução: $O(N^2 + NK)$ - dominada pela fase de contagem de caminhos. As restantes etapas apresentam complexidades assintoticamente inferiores que não afetam a complexidade dominante.

```
MAIN():
  Ler N, M, m1, m2, K
  Inicializar grafo com N nós

  Para i de 1 a K:
    Ler aresta (u, v)
    adj[u].adicionar(v)
    indeg[v]++

  ordem ← ORDENACAO_TOPOLOGICA(N, adj, indeg)

  Para cada origem s em ordem:
    Inicializar dp[] e visited[]
    dp[s] ← 1

    Para cada nó u em ordem:
      Para cada vizinho v de adj[u]:
        dp[v] ← (dp[v] + dp[u]) mod M

  Para cada destino t:
    camiao ← 1 + dp[t]
    Se m1 ≤ camiao ≤ m2:
      adicionarRota(s, t, camiao)

  ORDENAR_E_IMPRIMIR_ROTAS()
```

```
ORDENACAO_TOPOLOGICA(N, adj, indeg):
  fila ← nova Fila
  Para i de 1 a N:
    Se indeg[i] = 0:
      fila.inserir(i)

  Enquanto fila não vazia:
    u ← fila.remove()
    ordem.adicionar(u)
    Para cada v em adj[u]:
      indeg[v]--
      Se indeg[v] = 0:
        fila.inserir(v)

  Retornar ordem
```

Relatório 2º Projeto ASA 2025/2026

Grupo: AL030

Alunos: Leonor Costa Guedes (113396) e Manuel Francisco Santos Ramos Soares (113402)

Avaliação Experimental dos Resultados

Foram realizadas experiências com mais de dez instâncias de tamanho incremental, variando o número de nós N e de arestas K . Para cada instância foi calculado o valor da função de complexidade teórica $f(N, K) = N \times (N + K)$ e medido o tempo de execução correspondente. Os resultados obtidos encontram-se organizados numa tabela que apresenta, para cada instância, os valores de N , K , $f(N, K)$ e o tempo medido em segundos.

Com base nesses dados, foi gerado um gráfico onde o eixo dos XX representa a complexidade teórica prevista $f(N, K)$ e o eixo dos YY representa o tempo de execução. O gráfico inclui os pontos correspondentes aos tempos medidos e uma linha de tendência linear.

Observa-se uma relação aproximadamente linear entre o tempo de execução e o valor de $f(N, K)$. À medida que a complexidade teórica aumenta, o tempo cresce de forma proporcional, com pequenas variações explicáveis por fatores experimentais como o sistema operativo e a gestão de memória. A proximidade dos pontos à linha de tendência linear confirma que o comportamento empírico da implementação está de acordo com a análise teórica prevista, validando experimentalmente a complexidade $O(N \times (N + K))$.

N (nós)	K (arestas)	f(N,K)	Tempo (s)
100	1490	159000	0.013606
200	5852	1210400	0.011097
400	24030	9772000	0.038153
600	53826	32655600	0.083553
800	96001	77440800	0.144323
1000	149612	150612000	0.229977
1500	337093	507889500	0.556108
2000	599211	1202422000	1.410315
2500	936922	2348555000	2.330477
3000	1350568	4060704000	3.064860
3500	1834762	6433917000	4.753576
4000	2401263	9621052000	6.476883

